System Architecture Handover: Database Normalization & Presentation Layer Alignment

Document Classification: Technical Specification & Migration Blueprint

Target Audience: Database Engineers, System Architects, Large Language Models (LLMs) parsing system context.

Abstract

The current relational database schema (venue_rental_db.sql) exhibits structural deviations from the required data contracts established by the Presentation Layer (UI). To achieve seamless View-Controller-Model synchronization ($V \rightleftharpoons C \rightleftharpoons M$), structural normalization and state machine constraints must be strictly enforced. This document outlines the requisite schema modifications to ensure data integrity and $O(1)$ mapping complexity for the backend API.

Section I: Identity Entity Normalization (Users Table)

1.1 Architectural Deficit

The existing users table relies on a scalar role attribute (ENUM('User', 'Normal_Admin', 'Super_Admin')) to differentiate entity types. However, the UI requires domain-specific attributes that do not intersect. A unified table without these attributes results in null-pointer exceptions at the View layer.

1.2 Structural Refactoring Directives

The identity matrix must be decoupled or robustly expanded.

- **Attribute Injection (Student Domain):** The schema must support academic vectors.

$\Rightarrow$ Inject faculty (e.g., 'FCI', 'FOE') and course (e.g., 'BIT (Hons.)') columns.

- **State Machine Injection (Account Governance):**

A status column must be introduced to manage account lifecycles.

$\Rightarrow$ Define $S_{admin} \equiv \{\text{'Active'}, \text{'Suspended'}\}$.

$\Rightarrow$ Define $S_{student} \equiv \{\text{'Active'}, \text{'Suspended'}, \text{'Graduated'}\}$.

- **Role Constraint Enforcement (RBAC):**

Ensure programmatic or schema-level constraints to prevent the existence of multiple Super_Admin entities, maintaining $N_{SuperAdmin} \equiv 1$.

Section II: Finite State Machine (FSM) Alignment

2.1 Venue Operational States

The venues table must project a state vector that perfectly mirrors the UI dashboard toggles.

- **Current Vector:** status $\in$ {'Available', 'Maintenance', 'Closed'}.
- **Directive:** Ensure this ENUM definition is strictly preserved and enforced. No external states (e.g., 'Booked') should be recorded here, as temporal availability is a derivative of the bookings table.

2.2 Booking Transition States

The bookings table governs the core transaction lifecycle.

- **Required Vector:** The booking_status must accommodate the transition matrix presented in the approval queue.
- **Directive:** Ensure ENUM supports the fundamental subset $S \subset \{\text{'Pending'}, \text{'Approved'}, \text{'Rejected'}\}$ alongside post-execution states ($\text{'Completed'}, \text{'Cancelled'}$).

Section III: Relational Join Optimization (Data Matrix Mapping)

3.1 Foreign Key Integrity

The manage_bookings.php controller executes a topological projection requiring simultaneous access to Identity and Asset data:

$$Query_{booking} = Bookings \bowtie_{user_id} Users \bowtie_{venue_id} Venues$$

- **Directive:** Ensure standard INNER JOIN operations can map users.full_name and venues.venue_name against a single booking_id with sub-millisecond latency. ON DELETE CASCADE rules currently implemented must be carefully reviewed to prevent accidental deletion of historical ledger data when a student graduates.

Section IV: Polymorphic Financial Ledger Aggregation

4.1 Architectural Requirement

The report.php module aggregates financial velocity via a UNION ALL operation, merging base deposits with post-inspection penalties.

4.2 Schema Validation Requirements

To guarantee the mathematical union operates without casting errors, the underlying tables must possess symmetric data types:

- **Timestamp Synchronization:** payments.updated_at and inspections.inspected_at must both be queryable as standard TIMESTAMP or DATETIME formats.
- **Precision Alignment:** payments.deposit_paid and inspections.assessed_penalty must maintain identical precision (e.g., DECIMAL(10,2)).

Section V: Format Decoupling (Separation of Concerns)

5.1 Primary Key Projection Notation

The database schema **shall not** be altered to store string-prefixed IDs (e.g., EMP-0001, VEN-101).

- **Directive:** Maintain all primary keys (user_id, venue_id, booking_id) as INT(11) AUTO_INCREMENT.
- **Rationale:** The transformation mapping $f(x) \rightarrow \text{String}$ (utilizing CONCAT and LPAD) is strictly assigned to the PHP Controller layer. Modifying the database to store VARCHAR primary keys will exponentially degrade index traversal performance ($\text{Time Complexity: } O(\log n) \rightarrow O(n)$).

Appendix: Instructions for LLM / AI Assistants

Context Note for AI Processing this document:

When generating SQL ALTER TABLE scripts or PHP Data Access Objects (DAO) based on this blueprint, adhere strictly to the variable names and ENUM domains defined herein. Cross-reference these requirements with the provided data_template JSON/Array structures to ensure the output data contracts are mathematically identical to the required presentation formats. Do not alter the INT primary key constraints. Focus generated code on structural mutations and UNION query construction.